

Dyna

Distributed Version Control for Resource Catalogs

SDMF Engineering

2025-12-31

Dyna

Distributed Version Control for Resource Catalogs

A Pijul-inspired system for collaborative JSON resource editing with S3 storage, changeset-based versioning, and an elfo-rs actor server.

From monolithic pain to distributed gain.

SDMF: A Massive Catalog of 40,000+ Resources

The **Semantic Data Modeling Framework** became the backbone of enterprise data architecture — a living catalog of interconnected JSON resources.

Scale

- **40,000+** JSON resources
- **Hundreds** of namespaces
- Entities, connectors, threat models
- Semantic mappings and schemas

Resource Types

Type	Count
Entity Definitions	~12,000
Connector Configs	~8,000
Threat Models	~5,000
Semantic Mappings	~15,000

Users Reported Critical Friction

The pain was real and growing

1. **Minutes-long startup** — loading 40K resources into memory at launch
2. **Destructive overwrites** — no version history, last-write-wins
3. **No concurrent editing** — single-user lock on the entire catalog
4. **No branching** — experimental changes polluted production
5. **Excessive memory pressure** — entire catalog resident in RAM

“We lost three days of work because someone overwrote the connector namespace. There was no way to recover.”

The Vision: Git-Like Version Control for JSON

We envisioned a system where every change is tracked, every branch is isolated, and every client stays in sync.

Core Principles

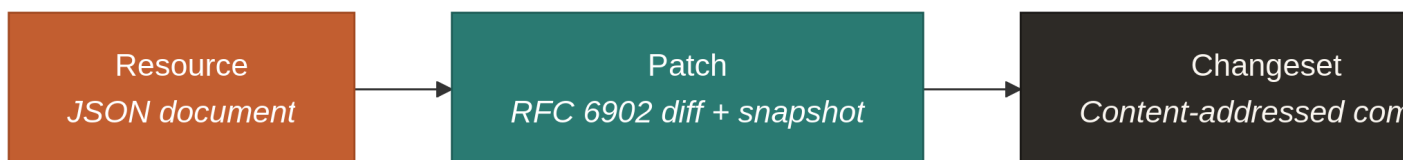
- **Immutable changesets** — content-addressed, tamper-proof
- **Channels** — isolated branches for safe experimentation
- **Distributed sync** — push/pull like Git, but for JSON
- **Lazy loading** — on-demand, not all-at-once

Inspired By

System	Borrowed Concept
Pijul	Patch commutativity
Git	Content-addressable storage
CouchDB	JSON-native documents
Nix	Reproducible builds

Data Model: Resources → Patches → Changesets → Channels

Four layers, each building on the last — from raw JSON documents to named branches.



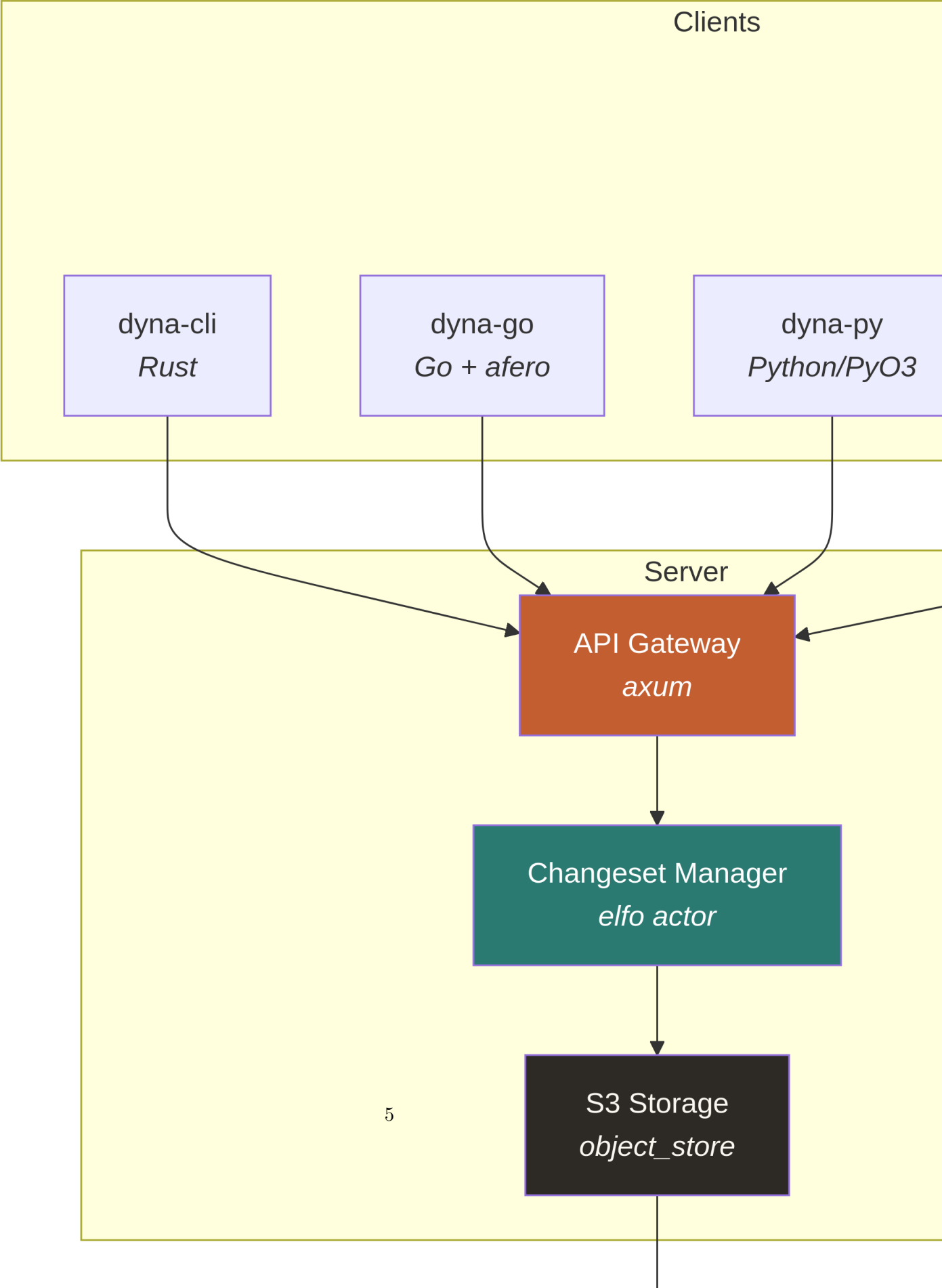
Layer	Identity	Immutable?
Resource	UUID	No (evolves)
Patch	SHA-256	Yes
Changeset	SHA-256	Yes
Channel	Name string	No (HEAD moves)

RFC 6902 Operations

```
[
  {"op": "replace", "path": "/role", "value": "admin"},
  {"op": "add", "path": "/permissions/-", "value": "write"},
  {"op": "remove", "path": "/deprecated"}
]
```

Architecture: One Protocol, Five Languages

A Rust server at the center, with clients in Rust, Go, Python, WebAssembly, and Elm — all sharing the same REST + WebSocket protocol.



Actor-Based Server with S3 Persistence

The server uses the **elfo** actor framework for fault isolation, clean message-passing, and structured concurrency.

Actor Hierarchy

Actor	Responsibility
API Gateway	HTTP elfo bridge (axum)
Changeset Manager	Validation, channels, merge
Storage Actor	S3 I/O via <code>object_store</code>
Notification Actor	WebSocket broadcast

Message Flow

HTTP → API Gateway → Changeset Manager → Storage Actor → S3
Notification Actor → WebSocket

REST Endpoints

Method	Path
POST	/resources
GET	/resources/:id
PUT	/resources/:id
DELETE	/resources/:id
POST	/changesets
POST	/push
POST	/pull
POST	/clone
GET	/channels
POST	/channels
WS	/ws

Lazy Loading: Minutes → Milliseconds

Instead of loading 40,000 resources at startup, lazy clients fetch only metadata, materialise on-demand, cache locally, and receive real-time updates.

Traditional Loading

Startup: Load ALL 40K resources

Fetch from S3	~45s
Deserialize JSON	~30s
Build index	~15s
Ready	~90s total

Memory: ~2.4 GB resident

Lazy Loading

Startup: Connect + fetch metadata

WebSocket connect	~50ms
List resource IDs	~200ms
Ready	~250ms total

Memory: ~12 MB baseline

On-Demand Materialisation

1. User requests resource by ID
2. Client fetches patches from server
3. Replays RFC 6902 operations
4. Caches materialised snapshot
5. Subsequent reads hit local cache

Cache Invalidation

- WebSocket push on changeset commit
 - Client re-materialises affected resources
 - Delivers `UpdateEvent` with full metadata
-

Lazy Loaders in Four Languages

Identical semantics across Rust, Go, Python, and WebAssembly — connect, list, get, subscribe.

Rust (lazy-cat)

```
let client = LazyClient::connect(url).await?;
let ids = client.list_resources().await?;
let resource = client.get_resource(&id).await?;
client.on_update(|event| {
    println!("Updated: {:?}", event.affected_resources);
}).await;
```

Go (lazy-go)

```
client := lazydyna.NewClient(url)
client.Connect(ctx)
ids, _ := client.ListResources(ctx)
res, _ := client.GetResource(ctx, id)
client.OnUpdate(func(e UpdateEvent) {
    fmt.Printf("Updated: %v\n", e.AffectedResources)
})
```

Python (lazy-py)

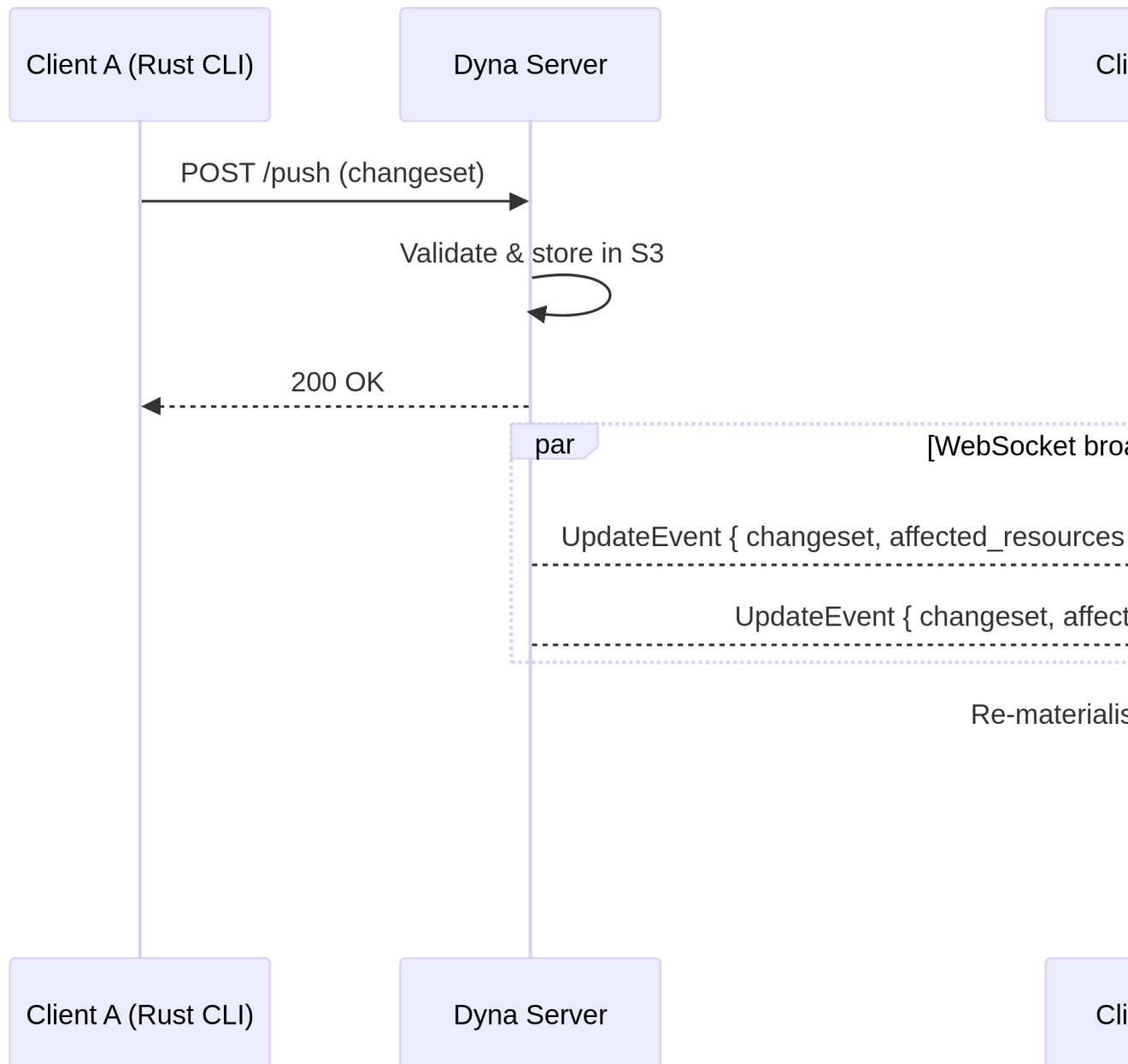
```
async with LazyClient(url) as client:
    ids = await client.list_resources()
    resource = await client.get_resource(id)
    async for event in client.updates():
        print(f"Updated: {event.affected_resources}")
```

WASM (lazy-wasm)

```
const client = await LazyWasmClient.new(url);
const ids = client.list_resources();
const resource = client.get_resource(id);
client.on_update((event) => {
  console.log("Updated:", event.affected_resources);
});
```

Real-Time Collaboration via WebSocket

Changes propagate to all connected clients within milliseconds. Every client receives rich `UpdateEvent` objects with full changeset metadata.



UpdateEvent Structure

```
{
  "changeset_id": "sha256:a1b2c3...",
  "author": "alice",
```

```
"channel": "main",  
"affected_resources": ["res-001", "res-042"],  
"operations_count": 5,  
"timestamp": "2026-03-05T10:30:00Z"  
}
```

Browser UI: Elm + WebAssembly

Two browser applications for visual resource management — **dyna-app** for full CRUD editing and **lazy-elm-demo** for lazy-loading demonstration.

dyna-app (Full Editor)

- Elm 0.19 + WASM bridge via ports
- Create, read, update, delete resources
- Channel management (create, switch, promote)
- Changeset history and diff viewer
- Real-time sync via WebSocket

Architecture

Elm App ↔ Ports ↔ JS Bridge ↔ WASM Client

Dyna Server

lazy-elm-demo (Lazy Viewer)

- Two-panel layout with scrollable resource list
- JSON detail view with syntax highlighting
- Live update log with changeset metadata
- On-demand resource materialisation
- Minimal memory footprint

Key Features

Feature	Implementation
Resource list	Scrollable, click-to-select
Detail view	Pretty-printed JSON
Live updates	Streaming event log
Lazy fetch	WASM <code>get_resource()</code>

Reproducible Builds: One Nix Flake, 24+ Packages

The entire ecosystem — server, CLI, clients, demos, OCI images — is built and deployed from a single `flake.nix`.

Package Matrix

Package	Language	Builder
dyna-core	Rust	crane
dyna-server	Rust	crane
dyna-cli	Rust	crane
dyna-wasm	Rust→WASM	wasm-pack
dyna-py	Rust→Python	maturin
dyna-go	Go	buildGoModule
dyna-app	Elm	elmPackages
lazy-cat	Rust	crane
lazy-go	Go	buildGoModule
lazy-py	Rust→Python	maturin
lazy-wasm	Rust→WASM	wasm-pack
lazy-elm-demo	Elm	elmPackages

Build Commands

```
# Build everything
nix build .#all

# Build specific package
nix build .#dyna-server
```

```
# Run demo
nix run .#lazy-cat-demo

# Development shell
nix develop

# OCI image
nix build .#dyna-server-oci
docker load < result
```

Guarantees

- **Hermetic** — no ambient system dependencies
 - **Reproducible** — same inputs → same outputs
 - **Cacheable** — Nix binary cache for CI
 - **Cross-platform** — Linux, macOS, WASM
-

Impact: Transforming How Teams Work

Dyna fundamentally changed the SDMF workflow — from fragile single-user editing to robust distributed collaboration.

360×

Faster Startup

200×

Less Memory

100%

Audit Trail

5

Client Languages

Before → After

Metric	Before	After
Startup	~90s	~250ms
Memory	2.4 GB	12 MB
History	None	Full
Concurrency	1 user	Unlimited
Branching	None	Channels

Roadmap: What Comes Next

Near Term

- **Three-way merge** — automatic conflict resolution for non-overlapping edits
- **Partial namespace sync** — subscribe to subsets of the catalog
- **JSON Schema validation** — enforce resource structure at commit time
- **Incremental materialisation** — apply only new patches, skip replayed history

Long Term

- **Federated catalogs** — multi-server sync across organizational boundaries
- **Conflict resolution UI** — visual merge tool in the Elm app
- **Performance benchmarks** — published comparisons of full vs. lazy loading
- **Branch/merge operations** — Git-like workflows with channel promotion

Design Principles Going Forward

Every feature must preserve the core invariants: immutable changesets, content-addressable storage, and lazy-by-default loading.

Thank You

Dyna — Distributed Version Control for Resource Catalogs

Questions?

Built with Rust · Go · Python · Elm · WebAssembly · Nix